CSE 109

Complete Revision Guide

Syntax · Common Codes · Reminders · Mnemonics · 35 Bit Tricks

What this is: the last-days revision book for all 5 chapters (C basics, arrays/strings, pointers/functions/recursion, structures/files, C++/OOP). No long questions — only what you must *remember and be able to write quickly*.

How to use:
1. Read the **Top 40 Reminders** and **Mnemonics** first (pages 2–4).
2. For each chapter: read the syntax boxes, then *cover* each common code and try to write it yourself.
3. The **Bitwise** chapter has 35 verified tricks — learn the table, the code is one line each.
4. Last chapter: output-tracing rules + error checklist + exam-day plan.

BUET — L-1/T-1 — EEE

All bit tricks and key programs verified with GCC / G++.

Contents

1 Top 40 Reminders and Mnemonics **3**
1.1 Top 40 exam reminders (read every day) . . . 3
1.2 Mnemonics . . . 3

2 C Basics: Types, Operators, I/O, Control, Loops **5**
2.1 Program skeleton and data types . . . 5
2.2 Operators . . . 5
2.3 printf / scanf quick sheet . . . 5
2.4 Decisions . . . 6
2.5 Loops . . . 6
2.6 Common number codes (write without looking) . . . 6
2.7 Pattern method . . . 7
2.8 math.h results to remember . . . 7

3 Bitwise Operations and Special Bit Tricks **8**
3.1 The six operators . . . 8
3.2 Two's complement and bit-pattern questions . . . 8
3.3 35 special bit tricks (all verified with GCC) . . . 8
3.4 Ready functions (past-paper style) . . . 9

4 Arrays, Matrices, Strings **10**
4.1 Array syntax . . . 10
4.2 1D array codes . . . 10
4.3 Sorting and searching . . . 10
4.4 Matrix codes . . . 10
4.5 String syntax and library . . . 11
4.6 Write-your-own string functions (asked very often) . . . 11
4.7 String problem codes . . . 11

5 Pointers, Functions, Recursion, Dynamic Memory, Macros **12**
5.1 Pointer syntax and arithmetic . . . 12
5.2 Reading declarations (right–left rule) . . . 12
5.3 Functions . . . 12
5.4 Recursion templates . . . 12
5.5 Dynamic memory . . . 13
5.6 Macros . . . 13

6 Structures, Unions, Enums, typedef, Bit-fields, Files **14**
6.1 Structure syntax . . . 14
6.2 Union, enum, bit-field . . . 14
6.3 File syntax . . . 14
6.4 File codes (a file question came in all 14 papers) . . . 14

7 C++ and Object Oriented Programming **16**
7.1 C++ basics . . . 16
7.2 Class skeleton with everything (Rule of Three) . . . 16

7.3 Constructors, destructors, static, friend . . . 16
7.4 Operator overloading reference . . . 17
7.5 Inheritance . . . 17
7.6 Virtual functions and abstract classes . . . 17

8 Output Tracing Rules, Error Checklist, Exam Plan **19**
8.1 Output-tracing rules (one line each) . . . 19
8.2 Find-the-error checklist . . . 19
8.3 Theory one-liners (quick definitions) . . . 19
8.4 Exam-day plan . . . 20

1 Top 40 Reminders and Mnemonics

1.1 Top 40 exam reminders (read every day)

1. Integer division cuts decimals: `7/2=3`, `1/2=0`. Use `1.0/k` or `(double)a/b`.
2. `%` only for integers; sign follows the left side: `-7%2 = -1`.
3. `=` assigns, `==` compares. `if(x=8)` is always true.
4. `&&` stops at the first false, `||` at the first true (short circuit).
5. `a++` use-then-add, `++a` add-then-use. `i++ * i++` = undefined.
6. Local variables start with **garbage**; globals and statics start with **0**.
7. `scanf` needs `&` — except for strings (array name is an address).
8. `double`: `%lf` in scanf; `%f` or `%lf` in printf.
9. `%010.4d`: precision given ⇒ 0-flag ignored ⇒ `" 0004"`.
10. `switch`: integer/char constants only; no `break` = fall through; ranges via `mark/10`.
11. `do{}while();` runs at least once and ends with `;`.
12. `break` leaves only the innermost loop.
13. Array index `0..n-1`; C never checks bounds.
14. Never `int a[n];` before reading n (VLA is banned in BUET answers).
15. A string needs one extra byte for `'\0'`.
16. `s1 = "abc"` is an error → `strcpy`. `s1 == s2` compares addresses → `strcmp`.
17. `strcmp` returns **0** when equal.
18. `char *p; scanf("%s",p);` → p points nowhere. Give memory first.
19. Pointer + 1 moves by `sizeof(type)`; `q-p` counts elements.
20. 2D array value needs 2 stars: `*(*(a+i)+j)`.
21. Never return the address of a local variable/array.
22. Every `malloc/new` needs `free/delete` (`new[]` → `delete[]`).
23. Free a dynamic 2D array: rows first, then the row-pointer array.
24. Recursion: base case first, then a smaller call.
25. Macro: bracket every parameter **and** the whole body; arguments may be evaluated twice.
26. Struct pointer uses `->`; struct variable uses `.`
27. Union size = largest member; only the last written member is valid.
28. Bit-field: no address (`&`), no arrays, no direct scanf.
29. Always check `fopen` for `NULL` and always `fclose`.
30. `"w"` erases, `"a"` appends, `"r"` must exist.
31. Read until end: `while((c=fgetc(fp))!=EOF)` with `int c`.
32. Windows paths: double backslash `"D:\\a.txt"`.
33. If you write any constructor, there is no free default constructor (`new A[5]` fails).
34. Class owns memory ⇒ destructor + copy constructor + `operator=` (Rule of Three).
35. Copy constructor runs on: `A b=a;`, pass by value, return by value. Form `A(const A &o)`.
36. Base constructor first, derived destructor first.
37. Private members are never accessible in derived classes.
38. Virtual call ⇒ object's version; non-virtual ⇒ pointer type's version. By value ⇒ sliced.
39. Base class with virtual functions needs a **virtual destructor**.
40. Postfix `operator++(int)` returns the old copy; `<<` must be a friend returning `ostream&`.

1.2 Mnemonics

Mnemonic — operator precedence

"Ugly **A**pes **S**teal **R**ed **B**ananas, **L**eaving **T**iny **A**pple **C**ores"
Unary → **A**rithmetic (`* / %` then `+ -`) → **S**hift → **R**elational (`< >` then `== !=`) → **B**itwise (`& ^ |`) → **L**ogical (`&& ||`) → **T**ernary → **A**ssignment → **C**omma.
Bitwise inner order: **"AXO"** — And, Xor, Or. Logical: `&&` before `||` ("AND is stronger").
Right-to-left only for: **U**nary, **T**ernary, **A**ssignment ("UTA goes backwards").

Mnemonic — printf format

`%[flags][width][.precision]type` = **"Fat Whales Prefer Tea"**. Flags: `-` left, `0` zeros, `+` sign. `.5s` = first 5 characters.

Mnemonic — pointers

"**& = Address of, * = value At**" (**A**nd = **A**ddress).
`*p++` "use, then move"; `(*p)++` "change the value"; `*++p` "move, then use".
"**Count the stars**": a *k*-D array name needs *k* stars to reach a value.

Mnemonic — misc

Two's complement: "Flip and add one". $\sim x = -x-1$.
Leap year: "4 yes, 100 no, 400 yes".
Default values: "Globals and Statics Get zero, locals Get Garbage".
Recursion: "**B**ase, **S**hrink, **C**ombine" (BSC).

Macro: "**B**racket **E**very **P**arameter and the **B**ody" (BEPB).
Files: "**r** = read (must exist), **w** = wipe, **a** = add at end; **+** = both ways".
strcmp: "Zero means Same". **Struct access:** "Arrow for pointers, Dot for objects".
Rule of Three: "**D**on't **C**opy **A**ccidentally" = **D**estructor, **C**opy constructor, **A**ssignment.
Construction order: "Base Builds first, Base Breaks last".
Inheritance access: "Public keeps, Protected shields, Private hides" (and private members are never inherited-accessible).
Virtual: "Virtual looks at the **Object**, non-virtual looks at the **Pointer**".
Copy constructor calls: "**I**nit, **P**ass, **R**eturn" (IPR).
OOP pillars: "**A PIE**" — Abstraction, Polymorphism, Inheritance, Encapsulation.
Patterns: "Spaces, Stars, What-at-j" — 3 questions per row.

2 C Basics: Types, Operators, I/O, Control, Loops

2.1 Program skeleton and data types

```c
#include <stdio.h>
#include <math.h>        // sqrt pow fabs ceil floor round   (compile with -lm on Linux)
#include <string.h>      // strlen strcpy strcmp strcat strstr
#include <ctype.h>       // isalpha isdigit isspace toupper tolower
#include <stdlib.h>      // malloc calloc realloc free abs atoi exit
#define PI 3.1416        // symbolic constant (no ;)
int main() {
    /* code */
    return 0;
}
```

Type	Bytes	Range	Fmt	Type	Bytes	Range	Fmt
`char`	1	−128..127	`%c %d`	`float`	4	6–7 digits	`%f`
`short`	2	±32767	`%hd`	`double`	8	15 digits	`%lf`
`int`	4	$\pm 2.1 \times 10^9$	`%d`	`long long`	8	$\pm 9.2 \times 10^{18}$	`%lld`
`unsigned`	4	$0..4.29 \times 10^9$	`%u`	`unsigned long long`	8	$0..1.8 \times 10^{19}$	`%llu`

Primary: char int float double void. **Derived:** array, pointer, function. **User-defined:** struct union enum typedef.
Names: letters, digits, _; not starting with a digit; no space/-/$; not a keyword; case sensitive.
`'a'` = char constant (1 byte, value 97) `"a"` = string (2 bytes: `'a'`, `'\0'`).
ASCII: `'0'`=48, `'A'`=65, `'a'`=97, space = 32, `'\n'` = 10. `'a'-'A'` = 32.

2.2 Operators

Level	Operators	Direction
1	`() [] -> . x++ x--`	→
2	`++x --x + - ! ~ (type) * & sizeof`	←
3 / 4	`* / %` then `+ -`	→
5	`<< >>`	→
6 / 7	`< <= > >=` then `== !=`	→
8 / 9 / 10	`&` then `^` then `|`	→
11 / 12	`&&` then `||`	→
13	`?:`	←
14	`= += -= *= /= %= <<= >>= &= ^= |=`	←
15	`,`	→

Exam trap / remember

`x & 1 == 0` means `x & (1 == 0)` — always bracket bit tests: `(x & 1) == 0`.
`a < b < c` is `(a<b) < c` (0/1 compared with c): `5 > 3 > 1` is 0.
`15 >> 4/2` = `15 >> 2` = 3 (division first).
Type of result: `int op int = int`; any `double` makes the whole result double. `int g = 7.9/2;` gives 3.

Promotion order: char/short → int → unsigned → long → long long → float → double. `unsigned u = 0; u - 1` = 4294967295.

2.3 printf / scanf quick sheet

`printf("%5d", 42)`	␣␣␣42	`printf("%-5d|", 42)`	42␣␣␣—
`printf("%05d", 42)`	00042	`printf("%+d", 42)`	+42
`printf("%.2f", 3.14159)`	3.14	`printf("%8.3f", 3.14159)`	␣␣␣3.142
`printf("%e", 4300.303)`	4.300303e+03	`printf("%g", 4.3030)`	4.303
`printf("%x %X %o", 255,255,8)`	ff FF 10	`printf("%.3s", "Bangla")`	Ban
`printf("%c", 65)`	A	`printf("%%")`	%

```c
scanf("%d %lf %c", &n, &x, &ch);    // note: " %c" (space) skips the leftover newline
scanf("%s", word);                  // one word, no &
scanf(" %[^\n]", line);             // whole line with spaces
fgets(line, 100, stdin);            // whole line, keeps '\n'
while (scanf("%d", &x) == 1) { }    // read until input ends
```

2.4 Decisions

```c
if (c1) { } else if (c2) { } else { }
max = (a > b) ? a : b;
max3 = (a > b) ? (a > c ? a : c) : (b > c ? b : c);     // single statement
switch (mark / 10) {                                    // range trick
    case 10: case 9: grade = 'A'; break;
    case 8:          grade = 'B'; break;
    default:         grade = 'F';
}
switch (a > b) { case 1: printf("%d", a); break; case 0: printf("%d", b); }   // "switch only"
switch (y % 4) { case 0: switch (y % 100) { case 0: /* y%400 */ break; default: /* leap */ } break; default: /* not */
     }
```

Mnemonic — grading tables

Ranges of 10 → `switch(mark/10)`. Ranges of 5 → `switch(mark/5)`. "No if-else" plus validity check → outer `switch(mark>=0 && mark<=100)`.

2.5 Loops

```c
for (i = 1; i <= n; i++) { }
while (cond) { }
do { } while (cond);                // ; !
// for -> do-while (safe conversion)
init; if (cond) do { body; update; } while (cond);
// leave nested loops
for (...) for (...) if (found) goto done;
done: ;
```

2.6 Common number codes (write without looking)

```c
// digits right to left / count / sum / reverse / palindrome
cnt = 0; do { cnt++; t /= 10; } while (t);                  // digits (0 -> 1)
sum = 0; for (t = n; t > 0; t /= 10) sum += t % 10;          // digit sum
rev = 0; for (t = n; t > 0; t /= 10) rev = rev * 10 + t % 10;   // reverse; palindrome: rev == n

// prime
int isPrime(int n) { if (n < 2) return 0;
    for (int d = 2; d * d <= n; d++) if (n % d == 0) return 0; return 1; }

// all primes <= n, all factors, perfect number
for (k = 2; k <= n; k++) if (isPrime(k)) printf("%d ", k);
for (i = 1; i <= n; i++) if (n % i == 0) printf("%d ", i);
s = 0; for (i = 1; i < n; i++) if (n % i == 0) s += i;   perfect = (s == n);

// Armstrong (n-digit)
d = digits(n); s = 0; for (t = n; t; t /= 10) { p = 1; for (j = 0; j < d; j++) p *= t % 10; s += p; }

// largest prime factor
for (p = 2; p * p <= n; p++) while (n % p == 0) { big = p; n /= p; }  if (n > 1) big = n;

// prime factorization "2^3 x 5^4"
for (p = 2; p <= n; p++) if (n % p == 0) { c = 0; while (n % p == 0) { n /= p; c++; } printf("%d^%d ", p, c); }

// GCD / LCM / factorial / power / Fibonacci
while (b) { t = a % b; a = b; b = t; }   lcm = x / g * y;
f = 1; for (i = 2; i <= n; i++) f *= i;
p = 1; for (i = 0; i < n; i++) p *= x;
a = 0; b = 1; for (i = 0; i < n; i++) { printf("%d ", a); t = a + b; a = b; b = t; }

// base conversion (print with array)
k = 0; do { d[k++] = n % b; n /= b; } while (n); while (k) printf("%d", d[--k]);
```

```
// leap year
leap = (y % 4 == 0 && y % 100 != 0) || (y % 400 == 0);

// second max without array
if (x > m1) { m2 = m1; m1 = x; } else if (x < m1 && x > m2) m2 = x;

// series: sign every 3rd negative / alternating / term from previous term
s += (k % 3 == 0 ? -1.0 : 1.0) / k;
s += sign * 1.0 / (k * k);  sign = -sign;
term = term * (-x * x) / ((2*k) * (2*k + 1));   // sin x
```

2.7 Pattern method

Mnemonic — patterns

For row `i`: (1) how many **spaces**? (2) how many **stars/numbers**? (3) what to print at position `j`?

```
for (i = 1; i <= n; i++) {                      // centred pyramid
    for (j = 1; j <= n - i; j++) printf(" ");
    for (j = 1; j <= 2 * i - 1; j++) printf("*");
    printf("\n");
}
if (i == 1 || i == n || j == 1 || j == n) '*' else ' '      // hollow square
if (j == i || j == n - i + 1) '*' else ' '                   // X shape
for (j = 1; j <= i; j++) printf("%d", j); for (j = i-1; j >= 1; j--) printf("%d", j);   // 12321
printf("%c", 'A' + (i + j - 2) % N);                          // letter wrap pattern
val = 1; for (j = 0; j <= i; j++) { printf("%d ", val); val = val * (i - j) / (j + 1); }  // Pascal
```

2.8 math.h results to remember

`ceil(5.00001)=6`, `floor(4.9999)=4`, `round(-1.5)=-2`, `ceil(-1.5)=-1`, `floor(-1.5)=-2`, `fabs(-4.557)=4.557`, `pow(2,10)=1024.0`, `sqrt(2)=1.414214`, `2*acos(0)` $= \pi$.

3 Bitwise Operations and Special Bit Tricks

3.1 The six operators

Op	Rule (per bit)	Example $12 = 1100$, $10 = 1010$	Use
`&`	1 if both 1	`12 & 10 = 1000 = 8`	clear / test bits (masking)
`|`	1 if any 1	`12 | 10 = 1110 = 14`	set bits
`^`	1 if different	`12 ^ 10 = 0110 = 6`	toggle, compare, swap
`~`	flip all	`~12 = -13`	masks, `~x = -x-1`
`<<`	shift left, 0 in	`12 << 1 = 24`	$\times 2^k$, build masks
`>>`	shift right	`12 >> 2 = 3`	$\div 2^k$, read bits

Mnemonic — the four basic bit jobs

"**Read with &, Set with —, Toggle with ^, Clear with & ~**" — always against the mask `(1 << k)`.

Mask building: `1 << k` (only bit k), `(1 << n) - 1` (n ones at the right), `~((1<<n)-1)` (all except the low n), `0xFF` (low byte).
Use `unsigned` for shifting right: for negative signed numbers `>>` usually copies the sign bit (`-1 >> 31 = -1`). Shifting by ≥ 32 bits is undefined.

3.2 Two's complement and bit-pattern questions

Method for "x and y are signed char = 43 and -23; write the bit patterns"

Positive: binary in 8 bits ($43 = 32+8+2+1 = 0010\,1011$).
Negative: binary of $|y|$ → flip → +1 ($23 = 0001\,0111 \to 1110\,1000 \to 1110\,1001$).
Do the operation bit by bit. Read a result starting with 1: flip, +1, put a minus sign.
Shortcut for ~: `~x = -x - 1` ($\sim 43 = -44$, $\sim(-23) = 22$).
Subtraction $A - B$**:** add A + (two's complement of B), drop the carry out.
Why two's complement: one zero only, subtraction = addition, no end-around carry.

Practice answers from past papers: 83 & $-108 = 16$, — = -41, ^ = -57, $\sim 83 = -84$, $\sim(-108) = 107$. 98, -82: 34, -18, -52, -99, 81. 89, -28: 64, -3, -67, -90, 27.

3.3 35 special bit tricks (all verified with GCC)

Task	Code	Example → result
1. Odd or even	`x & 1` (1 = odd)	13 → 1, 8 → 0
2. Multiply by 2^k	`x << k`	`5 << 3` → 40
3. Divide by 2^k ($x \geq 0$)	`x >> k`	`100 >> 2` → 25
4. Read bit k	`(x >> k) & 1`	13 (1101): k=2 → 1, k=1 → 0
5. Set bit k	`x | (1 << k)`	13, k=1 → 15
6. Clear bit k	`x & ~(1 << k)`	13, k=2 → 9
7. Toggle bit k	`x ^ (1 << k)`	13, k=0 → 12
8. Power of two?	`x && !(x & (x-1))`	16 → 1, 12 → 0
9. Count set bits	`while(x){x &= x-1; c++;}`	13 → 3
10. Clear lowest 1-bit	`x & (x - 1)`	12 (1100) → 8
11. Keep only lowest 1-bit	`x & -x`	12 → 4
12. Swap without temp	`a^=b; b^=a; a^=b;`	a=5, b=9 → a=9, b=5
13. Is negative (sign bit)	`(x >> 31) & 1`	$-7 \to 1$, $7 \to 0$
14. Absolute value	`m = x >> 31; (x ^ m) - m`	$-7 \to 7$
15. min / max without if	`y ^ ((x^y) & -(x<y))` = min; `x ^ ((x^y) & -(x<y))` = max	x=13, y=6 → 6, 13
16. Opposite signs?	`(a ^ b) < 0`	(5, -3) → 1, (5, 3) → 0
17. Multiple of 4 / x mod 2^k	`(x & 3) == 0` / `x & ((1<<k)-1)`	12 → 1, 14 → 0; 29 mod 8 → 5
18. Flip low n bits	`x ^ ((1 << n) - 1)`	13, n=3 → 10

Task	Code	Example → result
19. Extract n bits from pos p	`(x >> p) & ((1 << n) - 1)`	0xB6, p=2, n=3 → 5
20. Rotate left / right by k	`(x<<k)|(x>>(32-k))` / `(x>>k)|(x<<(32-k))`	0x12345678, k=8 → 34567812 / 78123456
21. Reverse 32 bits	loop: `r=(r<<1)|(x&1); x>>=1;`	1 → 0x80000000
22. Parity (odd no. of 1s)	`count % 2` or XOR all bits	13 → 1, 5 → 0
23. Case change of a letter	`c ^ 32` toggle, `c | 32` lower, `c & ~32` upper	'a' → 'A', 'G' → 'g', 'g' → 'G'
24. Equal?	`(a ^ b) == 0`	(42, 42) → 1
25. Low / high byte, swap bytes	`x & 0xFF`, `(x>>8) & 0xFF`, `(x<<8)|(x>>8)`	0x1234 → 34, 12, 3412
26. Print binary	`for(i=31;i>=0;i--) printf("%d",(x>>i)&1);`	10 → ...00001010
27. +1 / −1 without + or −	`-~x` / `~-x`	7 → 8 / 6
28. Average without overflow	`(a & b) + ((a ^ b) >> 1)`	2147483646, 2147483640 → 2147483643
29. Single non-repeated number	XOR all elements	{4,7,2,7,4} → 2
30. Gray code	`x ^ (x >> 1)`	5 → 7, 6 → 5
31. Next power of two	`x--; x|=x>>1; x|=x>>2; x|=x>>4; x|=x>>8; x|=x>>16; x+1`	13 → 16, 16 → 16
32. All ones (signed −1)	`-1 >> 31` stays −1 (sign copied)	−1
33. Swap bit m of x with y's	`x ^ ((((x>>m)&1) ^ ((y>>m)&1)) << m)`	x=5, y=2, m=1 → 7
34. Set low n bits to 1	`x | ((1 << n) - 1)`	13, n=4 → 15
35. Clear low n bits	`x & ~((1 << n) - 1)`	10, n=3 → 8

3.4 Ready functions (past-paper style)

```c
int countBits(unsigned x)       { int c = 0; while (x) { c += x & 1; x >>= 1; } return c; }
int evenParity(int n)           { return countBits((unsigned) n) % 2 == 0; }
int isPowerOf2(unsigned x)      { return x && !(x & (x - 1)); }
int isMultipleOf4(unsigned x)   { return (x & 3) == 0; }
int invertBits(int x, int n)    { return x ^ ((1 << n) - 1); }
int absoluteVal(int x)          { int m = x >> 31; return (x ^ m) - m; }
int isPositive(int x)           { return ((x >> 31) & 1) ^ 1; }      // 0 if negative
int isNegative(int x)           { return (x >> 31) & 1; }
int SwapAbit(int x, int y, int m) { return x ^ ((((x >> m) & 1) ^ ((y >> m) & 1)) << m); }
unsigned leftRotate(unsigned x, int n)  { n %= 32; return n ? (x << n) | (x >> (32 - n)) : x; }
unsigned rightRotate(unsigned x, int n) { n %= 32; return n ? (x >> n) | (x << (32 - n)) : x; }
void printBinary(unsigned x)    { for (int i = 31; i >= 0; i--) printf("%d", (x >> i) & 1); }
void printBinaryRec(unsigned x) { if (x >= 2) printBinaryRec(x / 2); printf("%u", x % 2); }  // no bit ops, no leading
    0s

unsigned reverseMSBBits(unsigned x, int n) {       // reverse top n bits only
    int low = 32 - n;
    unsigned top = (n == 32) ? x : x >> low, rest = (n == 32) ? 0 : x & ((1u << low) - 1), r = 0;
    for (int i = 0; i < n; i++) { r = (r << 1) | (top & 1); top >>= 1; }
    return (n == 32) ? r : (r << low) | rest;
}
// bit matching: positions where a and b have the same bit
while (a || b) { if ((a & 1) == (b & 1)) cnt++; a >>= 1; b >>= 1; }
```

Exam trap / remember

`x & 1 == 1` is wrong (== first). `1 << 31` on int is negative — use `1u << 31`. `~` on a char gives an int (promotion): print with %d to see the signed value.

4 Arrays, Matrices, Strings

4.1 Array syntax

```
int a[5] = {1, 2};           // {1,2,0,0,0}      int b[] = {1,2,3};   // size 3
int z[100] = {0};            // all zero         n = sizeof(b)/sizeof(b[0]);
int m[3][4];                 // rows x cols      int w[][3] = {1,2,3,4};   // 2 rows (only first size omitted)
int x[2][3] = {{1,2,3},{4,5,6}};   // = {1,2,3,4,5,6}   short rows padded with 0
void f(int a[], int n);      // = int *a  (changes reach the caller)
void g(int m[][20], int r, int c);   // column size required
&a[i] = base + i*size        &m[i][j] = base + (i*C + j)*size     (row-major)
```

4.2 1D array codes

```
for (i = 0; i < n; i++) scanf("%d", &a[i]);                          // read
mx = mn = a[0]; for (i = 1; i < n; i++) { if (a[i] > mx) mx = a[i]; if (a[i] < mn) mn = a[i]; }
pos = -1; for (i = 0; i < n; i++) if (a[i] == x) { pos = i; break; }  // linear search
for (i = 0, j = n - 1; i < j; i++, j--) { t = a[i]; a[i] = a[j]; a[j] = t; }   // reverse
for (i = k; i < n - 1; i++) a[i] = a[i + 1]; n--;                     // delete at k
for (i = n; i > k; i--) a[i] = a[i - 1]; a[k] = x; n++;               // insert at k
for (i = 0, k = 0; i < n; i++) if (a[i] != val) a[k++] = a[i]; n = k;   // delete all val
int cnt[101] = {0}; for (i = 0; i < n; i++) cnt[a[i]]++;             // frequency (small range)
for (r = 0; r < k; r++) { last = a[n-1]; for (i = n-1; i > 0; i--) a[i] = a[i-1]; a[0] = last; }  // rotate right k
for (i = 1; i < n; i++) if (a[i] < a[i - 1]) sorted = 0;              // sorted check
mean = sum / n;  var += (a[i]-mean)*(a[i]-mean);  var /= n;          // variance
median = n % 2 ? b[n/2] : (b[n/2 - 1] + b[n/2]) / 2.0;               // b sorted copy
```

4.3 Sorting and searching

```
// Bubble (after pass i the largest i are fixed at the end)
for (i = 0; i < n - 1; i++)
    for (j = 0; j < n - 1 - i; j++)
        if (a[j] > a[j + 1]) { t = a[j]; a[j] = a[j + 1]; a[j + 1] = t; }

// Selection (pass i puts the i-th smallest at i; easy to show states)
for (i = 0; i < n - 1; i++) { m = i;
    for (j = i + 1; j < n; j++) if (a[j] < a[m]) m = j;
    t = a[i]; a[i] = a[m]; a[m] = t; }

// Binary search (sorted!)  -- first occurrence version keeps searching left
lo = 0; hi = n - 1; ans = -1;
while (lo <= hi) { mid = (lo + hi) / 2;
    if (a[mid] == x) { ans = mid; hi = mid - 1; }
    else if (a[mid] < x) lo = mid + 1; else hi = mid - 1; }

// Merge two sorted arrays
while (i < m && j < n) r[k++] = (x[i] < y[j]) ? x[i++] : y[j++];
while (i < m) r[k++] = x[i++];   while (j < n) r[k++] = y[j++];

// Counting sort for 0/1/2      // two-key compare (struct): a after b if
c0 c1 c2 counts, then rewrite   //  a.dept > b.dept || (a.dept == b.dept && a.id > b.id)
```

Mnemonic — sorting

"**Bubble bubbles the biggest to the back; Selection selects the smallest to the front.**" Descending: flip > to <. Binary search: "**sorted or nothing**", $\log_2 n$ steps.

4.4 Matrix codes

```
for (i = 0; i < r; i++) for (j = 0; j < c; j++) t[j][i] = m[i][j];        // transpose (c x r)
for (i = 0; i < n; i++) for (j = i + 1; j < n; j++) swap(m[i][j], m[j][i]); // in-place (square)
d1 += m[i][i];  d2 += m[i][n-1-i];  if (n odd) subtract centre once          // diagonals
for (i = 0; i < n; i++) swap(m[i][i], m[i][n-1-i]);                        // interchange diagonals
sym = 1; if (m[i][j] != m[j][i]) sym = 0;                                   // symmetric
if (i > j && m[i][j]) upper = 0;  if (i < j && m[i][j]) lower = 0;          // triangular / diagonal
zeros > n*n - zeros  -> sparse
for (i..r1) for (j..c2) { C[i][j] = 0; for (k..c1) C[i][j] += A[i][k] * B[k][j]; }   // c1 == r2
for (k = 0; k < n*n; k++) scanf("%d", &M[k / n][k % n]);                    // no nested loop
```

```
// sort 2D row-major: bubble on k = 0..R*C-1 using m[k/C][k%C]
// magic square: every row, column, both diagonals = n(n*n+1)/2 and each 1..n*n once
```

4.5 String syntax and library

```
char s[20] = "Hi";  char t[] = "Hello";  /* 6 bytes */  char *p = "const text";
char names[100][31];                 // array of strings       names[i]
char *list[100];                     // array of pointers (exact sizes via malloc)
```

`strlen(s)`	length	`strcmp(a,b)`	0 equal, <0 a first
`strcpy(d,s)`	copy	`strncpy(d,s,n)`	copy n
`strcat(d,s)`	append	`strstr(s,t)`	pointer to t in s / NULL
`strchr(s,c)`	first c / NULL	`atoi("12")`, `atof`	string → number
`isalpha isdigit isalnum`	ctype tests	`isspace ispunct isupper`	ctype tests
`toupper tolower`	convert	`sprintf(buf,"%d",n)`	number → string

4.6 Write-your-own string functions (asked very often)

```
int  myStrlen(char *s)            { char *p = s; while (*p) p++; return p - s; }
void myStrcpy(char *d, char *s)   { while ((*d++ = *s++)); }
int  myStrcmp(char *a, char *b)   { while (*a && *a == *b) { a++; b++; } return *a - *b; }
char *myStrcat(char *d, char *s)  { char *p = d; while (*p) p++; while ((*p++ = *s++)); return d; }
int  myStrstr(char s[], char t[]) {                 // index or -1, "" -> 0
    if (!t[0]) return 0;
    for (int i = 0; s[i]; i++) { int j = 0;
        while (t[j] && s[i + j] == t[j]) j++;
        if (!t[j]) return i; }
    return -1; }
void myStrrev(char *s)            { char *L = s, *R = s, t; while (*R) R++; R--;
                                    while (L < R) { t = *L; *L++ = *R; *R-- = t; } }
```

4.7 String problem codes

```
for (i = 0, j = len-1; i < j; i++, j--) if (s[i] != s[j]) pal = 0;         // palindrome
if (s[i] >= 'a' && s[i] <= 'z') s[i] -= 32;                                 // to upper
for (i = 0; s[i]; i++) if (s[i] != ' ' && (i == 0 || s[i-1] == ' ')) words++;   // words
if (i == 0 || s[i-1] == ' ') upper else lower                               // Name Case
int cnt[256] = {0}; for (i = 0; s[i]; i++) cnt[(unsigned char)s[i]]++;    // char frequency
for (i = 0, k = 0; s[i]; i++) if (s[i] != ch) s[k++] = s[i]; s[k] = '\0';  // remove char
last = -1; for (...) if (s[i]==ch) last = i; then shift left from last      // remove last occurrence
seen[256]: keep first occurrence only                                       // remove duplicates
for (j = 0; T[j] && S[i]; i++) if (S[i] == T[j]) j++;  sub = !T[j];         // subsequence
for (i = 0; s[i]; i++) val += (v(s[i]) < v(s[i+1])) ? -v(s[i]) : v(s[i]);  // Roman -> int
cnt[c]++ for word 1, cnt[c]-- for word 2, all zero -> anagram
expand(L, R) while s[L] == s[R]: longest palindromic substring
find smallest i with s[i..] palindrome, append reverse of s[0..i-1]         // make palindrome
reverse each word between spaces; or print words from the end               // word reversals
num = num*10 + (s[i]-'0'); parts==4, 0..255, no "01"                        // IPv4 check
for (i = 0; i < len && in[start+i]; i++) out[i] = in[start+i]; out[i] = 0;  // substring
```

Exam trap / remember

Reading a line after a number: `scanf("%d", &n);` leaves '\n' ⇒ use `scanf(" %[^\n]", s)` or `getchar()` first. `fgets` keeps the newline: remove with `s[strcspn(s,"\n")] = 0;`

5 Pointers, Functions, Recursion, Dynamic Memory, Macros

5.1 Pointer syntax and arithmetic

```
int x = 10, *p = &x, **pp = &p;      // *p == 10, **pp == 10, *p = 20 changes x
int *q = NULL;                       // safe empty pointer
p + 1  -> next element (+sizeof(type) bytes)    q - p -> elements between
a[i] == *(a + i)      &a[i] == a + i      p[-2] == *(p - 2)
```

Expr.	Meaning	Expr.	Meaning
`*p++`	value, then move p	`(*p)++`	increase value
`*++p`	move, then value	`++*p`	increase value, then use
`char* +1`	+1 byte	`int*/float* +1`	+4 bytes
`double* +1`	+8 bytes	`&p`	address of the pointer itself

2D (int a[R][C], base B, 4-byte int): `a+i` = row i = $B + iC \cdot 4$; `*(a+i)+j` = &a[i][j]; `*(*(a+i)+j)` = a[i][j]; `*a` = B (address), `**a` = a[0][0].
3D (int a[X][Y][Z]): 3 stars for a value; `*(*a+1)+3` = address of a[0][1][3].

5.2 Reading declarations (right–left rule)

`int *a[10]`	array of 10 pointers	`int (*a)[10]`	pointer to array of 10
`int *f(int)`	function returning int*	`int (*f)(int)`	pointer to function
`int *(*f)(int)`	ptr to fn returning int*	`void* f(int *a[])`	fn taking int**, returning void*

```
int sq(int x) { return x * x; }
int (*fp)(int) = sq;   fp(5);                    // 25
double (*op[4])(double, double) = {add, sub, mul, dvd};   op[k](a, b);   // menu calculator
```

5.3 Functions

```
int add(int a, int b);                // prototype (declaration)      a, b = formal parameters
s = add(x, 3);                        // call                          x, 3 = arguments (actual)
void swap(int *a, int *b) { int t = *a; *a = *b; *b = t; }   swap(&x, &y);   // call by reference
void stats(int a[], int n, int *mn, int *mx, double *avg);    // return many values
void counter() { static int c = 0; c++; }                      // static local keeps value
```

Kind	Scope	Lifetime	Default	Keyword
local	block	block	garbage	(auto)
static local	block	whole program	0	`static`
global	file (below it)	whole program	0	–
extern	refers to global elsewhere	–	–	`extern`

Scope types: block, function (`goto` labels), file, function-prototype. Declaration (no memory, `extern int x;`) vs definition (memory, `int x;`).

5.4 Recursion templates

Mnemonic — recursion

"**B**ase, **S**hrink, **C**ombine": stop case → call on a smaller input → build the answer. Trace: go **down** writing calls, come **up** filling values. Code *before* the call prints going down, *after* the call prints coming up.

```
int fact(int n)        { return n <= 1 ? 1 : n * fact(n - 1); }
int fib(int n)         { return n <= 1 ? n : fib(n - 1) + fib(n - 2); }
int power(int x, int n){ if (!n) return 1; int h = power(x, n/2); return n%2 ? h*h*x : h*h; }
int gcd(int a, int b)  { return b == 0 ? a : gcd(b, a % b); }
int sumDigits(int n)   { if (n < 0) n = -n; return n == 0 ? 0 : n % 10 + sumDigits(n / 10); }
void printDigits(int n){ if (n >= 10) printDigits(n / 10); printf("%d\n", n % 10); }
```

```
void printRev(char *s) { if (!*s) return; printRev(s + 1); putchar(*s); }
int count0(int a[], int n)  { return n == 0 ? 0 : (a[n-1] == 0) + count0(a, n - 1); }
int maxArr(int a[], int n)  { if (n == 1) return a[0]; int m = maxArr(a, n-1); return a[n-1] > m ? a[n-1] : m; }
int vowels(char *s)    { if (!*s) return 0; return (strchr("aeiouAEIOU", *s) != 0) + vowels(s + 1); }
int isPal(char *s, int i, int j) { return i >= j ? 1 : (s[i] == s[j] && isPal(s, i+1, j-1)); }
int bs(int a[], int lo, int hi, int x) { if (lo > hi) return -1; int m = (lo+hi)/2;
    return a[m]==x ? m : a[m] < x ? bs(a, m+1, hi, x) : bs(a, lo, m-1, x); }
int isPrimeR(int n, int d) { return d*d > n ? 1 : n % d == 0 ? 0 : isPrimeR(n, d + 1); }  // call (n,2), n>=2
void hanoi(int n, char f, char t, char v) { if (!n) return;
    hanoi(n-1, f, v, t); printf("disk %d %c->%c\n", n, f, t); hanoi(n-1, v, t, f); }  // 2^n - 1 moves
void gen(int d, int num, int r) { if (r == d) { printf("%d ", num); return; }
    for (int k = 1; k <= 3; k++) gen(d, num*10 + k, r + 1); }              // all d-digit from {1,2,3}
int sumZero(int *a, int n, int s, int i) { if (i == n) return s == 0;
    return sumZero(a, n, s + a[i], i+1) || sumZero(a, n, s - a[i], i+1); }  // +/- choices
void perm(int at, int n) { if (at == n) { print arr; return; }               // permutations
    for (i = 0; i < n; i++) if (!used[i]) { used[i]=1; arr[at]=i; perm(at+1,n); used[i]=0; } }
int isEven(int n) { return n == 0 ? 1 : isOdd(n - 1); }  int isOdd(int n) { return n == 0 ? 0 : isEven(n - 1); }
```

Recognise the trace: `func(x,y)= x + func(x,y-1)` $\Rightarrow x \times y$. `fun(a+a,b/2)(+a if b odd)` $\Rightarrow a \times b$.

5.5 Dynamic memory

```
int *a = (int *) malloc(n * sizeof(int));     // garbage values, NULL if fails
int *b = (int *) calloc(n, sizeof(int));      // zeros
a = (int *) realloc(a, 2 * n * sizeof(int));  // grow, keeps old values
free(a); a = NULL;

int **m = (int **) malloc(r * sizeof(int *));           // 2D: r row pointers
for (i = 0; i < r; i++) m[i] = (int *) malloc(c * sizeof(int));
for (i = 0; i < r; i++) free(m[i]);  free(m);           // rows first

double *ptr[12]; ptr[i] = malloc(days[i] * sizeof(double));   // jagged: exact sizes
int *process(...) { count k; res = malloc((k+1)*sizeof(int)); res[0] = k; fill; return res; }
```

Memory leak = allocated, never freed, address lost. **Dangling pointer** = points to freed/dead memory. **Returning arrays:** malloc it, or let the caller pass it, or `static`.
DMA advantages: size at run time, exact size, resizable, survives function return, large blocks.

5.6 Macros

```
#define SQ(x)  ((x) * (x))                        // bracket every parameter + whole body
#define MAX(a,b) ((a) > (b) ? (a) : (b))
#define AREA(r) (PI * (r) * (r))
#define ISDIGIT(c) (((c) >= '0' && (c) <= '9') ? 1 : 0)
#define RANGE(i,lo,hi) ((((i) >= (lo)) && ((i) <= (hi))) ? 1 : 0)
#define ROUND(x) ((int)((x) >= 0 ? (x) + 0.5 : (x) - 0.5))
#define test(i,x) (((i) % 2) ? (x) : (x) / 2)       // function -> macro: if/else -> ?:
```

Exam trap / remember

`#define cube(x) x*x*x`: `cube(9+1)` $= 9+9+9+1 = 28$. `#define MUL(a,b) a*b`: `100/MUL(2,5)` $= 250$. `abs(f())` calls f twice. No space between name and `(`, no `;` at the end.

Macro vs function: macro = text replacement, faster, no type check, code grows, args evaluated many times. Function = real call, type-checked, args evaluated once, one copy.

6 Structures, Unions, Enums, typedef, Bit-fields, Files

6.1 Structure syntax

```
struct Student { char name[30]; int roll; float cgpa; };      // ; at the end!
struct Student s = {"Rafi", 101, 3.85}, list[100], *p = &s;
strcpy(s.name, "Ana");   s.roll = 5;   p->cgpa = 3.9;   (*p).roll == p->roll
typedef struct { double x, y; } Point;   Point a = {1, 2};     // no 'struct' needed
struct Circle { Point c; double r; };    k.c.x = 3;            // nested
struct Student *arr = (struct Student *) malloc(n * sizeof(struct Student));
void update(struct Student *p) { p->cgpa += 0.1; }          // pass pointer: changes original, no copy
t = list[j]; list[j] = list[j+1]; list[j+1] = t;           // swap whole structures
```

Exam trap / remember

Structures can be assigned (`a = b`) but not compared (`a == b` is an error). `s.name = "x"` is an error. `*p.x` = `*(p.x)` (error for a struct pointer); `*p->x` = `*(p->x)` (x must be a pointer).

6.2 Union, enum, bit-field

```
union U { int i; char c; double d; };      // sizeof = 8 (largest); last written member valid
union { unsigned short n; unsigned char b[2]; } u;   swap u.b[0], u.b[1]  -> bytes swapped
enum Color { RED, GREEN, BLUE };           // 0, 1, 2       enum Month { JAN = 1, FEB, ... };
struct Date { unsigned day : 5; unsigned month : 4; unsigned year : 12; };   // 21 bits
```

	struct	union
memory	each member separate	all share one block
size	$\geq$ sum (padding!)	largest member
valid members	all	last written only

Bits for max value M: smallest k with $2^k > M$: 31→5, 12→4, 23→5, 59→6, 2050→12. **Padding example:** `struct{char c; int i; double d;}` = 1+3+4+8 = 16 (13 if "no padding"). **typedef uses:** short names for long types; portability (change type in one line); function-pointer types.

6.3 File syntax

Mode	Does	If missing	Mode	Does	If missing
`r`	read	NULL	`r+`	read+write, keep	NULL
`w`	write, **erase**	create	`w+`	read+write, erase	create
`a`	write at end	create	`a+`	read + append	create

Add `b` for binary (`"rb"`, `"wb"`). Text = characters (readable, newline translated). Binary = raw memory bytes (exact copy, `fread/fwrite`).

6.4 File codes (a file question came in all 14 papers)

```
FILE *in = fopen("input.txt", "r"), *out = fopen("output.txt", "w");
if (in == NULL || out == NULL) { printf("File error\n"); return 1; }

int c; while ((c = fgetc(in)) != EOF) { fputc(c, out); }                // char copy / count / case change
while (fscanf(in, "%d %s %f", &id, name, &m) == 3) { fprintf(out, ...); } // records
char line[200]; while (fgets(line, sizeof line, in)) { }                 // lines
fgets(line, sizeof line, in);                                            // skip header line
while (fscanf(f1, "%d", &a) == 1 && fscanf(f2, "%d", &b) == 1) fprintf(out, "%d\n", a + b);
while ((n = fread(buf, 1, sizeof buf, in)) > 0) fwrite(buf, 1, n, out);   // binary copy
fwrite(&s, sizeof s, 1, fp);   while (fread(&s, sizeof s, 1, fp) == 1) { }    // struct records
fseek(in, 0, SEEK_END); size = ftell(in);                                // file size
for (pos = size - 1; pos >= 0; pos--) { fseek(in, pos, SEEK_SET); fputc(fgetc(in), out); }  // reverse copy
long cnt = 0; while (fgetc(in) != EOF) cnt++;                            // size without fseek
fclose(in); fclose(out);
```

```c
// word frequency
while (fscanf(in, "%49s", w) == 1) { w[0] = tolower(w[0]); L = strlen(w);
    if (ispunct(w[L-1])) w[L-1] = 0;
    for (i = 0; i < cnt; i++) if (!strcmp(kw[i].word, w)) { kw[i].freq++; break; }
    if (i == cnt) { strcpy(kw[cnt].word, w); kw[cnt++].freq = 1; } }

// remove /* */ comments: state inComment; on '/', peek next with fgetc, push back with ungetc
// replace word in file: read all (fread) -> build new text with strncmp -> reopen "w" -> fputs
// garble pairs: read all into s[], swap s[i], s[i+1], rewrite with "w"
// dates with 30-day months: dayNo = Y*360 + M*30 + D ; gap = today - dayNo
```

Mnemonic — file program skeleton

"**O**pen, **C**heck, **L**oop, **W**rite, **C**lose" (OCLWC). Loop condition = the read call itself (`!= EOF`, `== n`, `!= NULL`).

7 C++ and Object Oriented Programming

7.1 C++ basics

```
#include <iostream>
#include <string>
#include <cstring>
#include <cmath>
using namespace std;                          // or std::cout
cin >> a >> b;   cout << "x = " << x << endl;
string s = "BUET";  s.length();  s + "EEE";  s == t;
int &r = x;                                   // reference = alias, must be initialised
void swapArgs(int &a, int &b) { int t = a; a = b; b = t; }      // swapArgs(x, y);
int *p = new int(5);  int *a = new int[n];  delete p;  delete[] a;
inline int sq(int x) { return x * x; }
void f(int a, int b = 10);                    // defaults from the right end only
template <class T> T getMax(T a, T b) { return a > b ? a : b; }
```

Pointer vs reference: reference must be initialised, never NULL, cannot re-seat, no `*`. `p2 = p1` copies an address; `r2 = r1` copies the object's contents.
new vs malloc: no size/cast, calls constructor/destructor, can initialise, overloadable, type-safe.
Overloading ambiguity: `f(float)/f(double)` with `f(10)`; `g(int)/g(int,int=0)` with `g(1)`; `h(int)/h(int&)` with `h(x)`. Return type alone cannot overload.
Inline: saves call overhead; not for big/recursive/loop functions; bodies inside a class are automatically inline.

7.2 Class skeleton with everything (Rule of Three)

```
class Str {
private:
    char *p;
    static int count;                                 // shared by all objects
public:
    Str() { p = new char[1]; p[0] = '\0'; count++; }                    // default
    Str(const char *s) { p = new char[strlen(s) + 1]; strcpy(p, s); count++; }   // parameterized
    Str(const Str &o) { p = new char[strlen(o.p) + 1]; strcpy(p, o.p); count++; }  // copy (deep)
    Str& operator=(const Str &o) {                                      // assignment (deep)
        if (this != &o) { delete[] p; p = new char[strlen(o.p) + 1]; strcpy(p, o.p); }
        return *this;
    }
    ~Str() { delete[] p; count--; }                                     // destructor
    Str operator+(const Str &b) const {                                 // concatenation
        Str r; delete[] r.p; r.p = new char[strlen(p) + strlen(b.p) + 1];
        strcpy(r.p, p); strcat(r.p, b.p); return r;
    }
    bool operator==(const Str &b) const { return strcmp(p, b.p) == 0; }
    const char *get() const { return p; }
    static int getCount() { return count; }                             // no 'this'
    friend ostream& operator<<(ostream &out, const Str &s) { return out << s.p; }
};
int Str::count = 0;                                                     // define static outside
```

Mnemonic — copy constructor

Called on "**I**nit, **P**ass, **R**eturn": `A b = a;`, pass by value, return by value. NOT on `b = a;` (that is `operator=`). Parameter must be `const A &` (by value would recurse forever).

Exam trap / remember

No copy constructor + pointer member ⇒ shallow copy ⇒ shared memory, changes leak to the other object, **double delete** when both destructors run. Fix: deep copy, or pass by reference. A class with only plain values (Point, Vector, Fraction) needs no copy constructor.

7.3 Constructors, destructors, static, friend

```
Rect() : w(1), h(1) {}                        // initializer list
Rect(double w, double h) { this->w = w; this->h = h; }
Demo(int a = 0) {}                            // also acts as default -> new Demo[5] works
```

```
A *arr = new A[5]; delete[] arr;                // 5 constructors, 5 destructors
box b[2][2] = {{box(1,2,3), box(2,3,4)}, {box(3,4,5), box(4,5,6)}};   // no default ctor
class Box { double l; friend double volume(Box b); };   double volume(Box b) { return b.l; }
class B;  class A { public: bool f(B b); };  class B { int v; friend bool A::f(B b); };   // member-friend
```

Order: objects destroyed in reverse order of creation. Private constructor ⇒ objects only from inside the class (factory/singleton). Static member function cannot use non-static members.

7.4 Operator overloading reference

```
T  operator+(const T &b) const;            // binary member: 1 parameter (left = *this)
friend T operator*(const T &a, const T &b);// binary friend: 2 parameters
T  operator-() const;                      // unary member: 0 parameters
T& operator+=(const T &b);                 // return *this by reference
T& operator++();                           // prefix  ++a
T  operator++(int);                        // postfix a++ : T old = *this; change; return old;
bool operator==(const T &b) const;   bool operator<(const T &b) const;   bool operator>(...)
T& operator=(const T &b);                  // check this != &b, return *this
int& operator[](int i);                    // return reference so a[i] = 5 works
friend ostream& operator<<(ostream &out, const T &t);   // return out
T operator+(double d) const;               // Complex + 1.5
```

Formula	Code
Complex $\times$	`(a.re*b.re - a.im*b.im, a.re*b.im + a.im*b.re)`
Complex $\div$	`den = c*c + d*d; ((a*c+b*d)/den, (b*c-a*d)/den)`
Complex $<$ (by modulus)	compare `re*re + im*im` (no sqrt)
Fraction $\pm$	`(n1*d2` $\pm$ `n2*d1, d1*d2)` then reduce by GCD, sign in numerator
Polar $\times$ / $\div$	multiply/divide r, add/subtract θ; dot = $r_1 r_2 \cos(\theta_1 - \theta_2)$
Vector cross	`(y*b.z - z*b.y, z*b.x - x*b.z, x*b.y - y*b.x)`

Restrictions: no new operators; cannot change precedence/associativity/arity; cannot overload `.` `::` `.*` `?:` `sizeof`; one operand must be a class; `=` `()` `[]` `->` members only; no default arguments.

7.5 Inheritance

```
class Car : public Vehicle { public: Car(int y, int d) : Vehicle(y), doors(d) {} };   // chaining
class Copier : public Printer, public Scanner { };   c.Printer::status();          // multiple, clash
class Student : virtual public Person {};  class Employee : virtual public Person {}; // diamond fix
class TA : public Student, public Employee { public: TA(string n) : Person(n) {} };
class Der : public Base { public: using Base::log; void log(string m); };          // undo name hiding
```

Base member →	public	protected	private
`: public B`	public	protected	hidden
`: protected B`	protected	protected	hidden
`: private B`	private	private	hidden

From `main`: only public members of the object's own (final) access level. Base doesn't know derived members.

7.6 Virtual functions and abstract classes

```
class Shape { public: virtual double area() = 0;  virtual ~Shape() {} };      // abstract
class Circle : public Shape { double r; public: Circle(double r) : r(r) {}
    double area() override { return 3.1416 * r * r; } };
Shape *s = new Circle(2);  s->area();  delete s;          // Circle::area, ~Circle then ~Shape
vector<unique_ptr<Shape>> v; v.push_back(make_unique<Circle>(1));   // auto delete
ostream& operator<<(ostream &o, const Shape &s) { s.print(o); return o; }   // virtual print helper
```

Mnemonic — which function runs?

Call through **pointer/reference**: virtual ⇒ **object's** version; non-virtual ⇒ **pointer type's** version. Call on an object passed **by value** to a base parameter ⇒ **sliced** ⇒ base version, and the copy's destructor prints. Non-virtual base destructor ⇒ `delete basePtr` runs only the base destructor (leak).

Overloading vs overriding: same name different parameters (compile time) vs same signature in derived class (run time, virtual).
OOP pillars (A PIE): Abstraction, Polymorphism, Inheritance, Encapsulation (private data + public functions).

8 Output Tracing Rules, Error Checklist, Exam Plan

8.1 Output-tracing rules (one line each)

1. Make a table of every variable; update one line at a time.
2. Integer division first, then the cast: `float f = 1/2;` → 0.000000.
3. `%d` with a double → garbage (undefined).
4. Uninitialised local → write "garbage".
5. Short circuit: in `a && b++`, if a is 0, b is unchanged.
6. `m = ++i && ++j && ++k`: stop at the first 0.
7. `while(i+1 ? --i : j++)`: evaluate the ternary each round.
8. Inner block variable with the same name hides the outer one (`int i = 2;` inside a for → prints 3 ten times, then 10).
9. Inside a function, a name not declared locally is the **global** one.
10. `switch` without `break`: execute all following cases.
11. `continue` in for still runs the update.
12. Loop that stops changing its variable → infinite loop.
13. `unsigned` overflow wraps: `(n+1)` with $n = 2^{32} - 1$ is 0 before any cast.
14. `%o` octal, `%x` hex, `%e` scientific, `%g` shortest.
15. Pointer printed with `%d`: base + index × size.
16. `*p++` prints then moves; `(*p)++` changes the array.
17. 3D index = $i \cdot (YZ) + j \cdot Z + k$.
18. Static local: initialised once, counts calls.
19. Macro: expand the text first, then apply precedence.
20. Recursion: write the call tree; print-before vs print-after.
21. Union: only the last member written is correct; sizeof = largest.
22. C++: `new A[5] + delete[]` → 5 constructor / destructor lines.
23. Derived object: base constructor line first, then derived.
24. Pass object by value → copy constructor line + destructor line at function end.
25. Virtual through pointer → object's version; slicing by value → base version.
26. `cout << f()`: f's own prints come before its returned value.
27. Default argument `b = x` uses the current global x (0 if never set).
28. Static class member is shared: changing it via one object changes all.

8.2 Find-the-error checklist

Look for	Fix
`int a[n];` before reading n	fixed size or malloc after reading
result printed inside the search loop	flag variable, print after the loop
`i` undeclared / loop from 0 using `a[i-1]`	declare; start at 1
max initialised to garbage or wrong comparison	init to `a[0]` or 0; check `>`
`p.name = "x"`	`strcpy`
struct pointer with `.` / brace-initialised pointer	`->`; point to a real struct
`char *p; gets(p);` / `scanf("%s",p)`	`char p[100]` or malloc
`scanf("%d", &pX)` with `int *pX`	`int x; pX = &x; scanf("%d", pX)`
returning a local array	malloc / caller's array / static
function returns `int` but gives an address	return type `int *`
macro without brackets	bracket params + body
`-tf()`, `muclass()`	`~tf()`, correct class name
missing `using namespace std;`	add it or use `std::`
`cin <<` / `cout >>`	`cin >>`, `cout <<`
class with pointer, no copy ctor, passed by value	deep copy ctor or pass by reference
`new Demo[5]` without default ctor	add `Demo()` or default argument
static member function using non-static data	make it non-static
friend uses a class declared later	forward declaration `class X;`
`%` with a double operand	compile error; use `fmod` or ints
`while(i>10)` with i = 0 / `k<1` limits	fix the loop bound

8.3 Theory one-liners (quick definitions)

Compiler: translates the whole program, then runs. **Interpreter:** line by line.
Von Neumann: Input, Output, Memory (program + data together), CPU = CU + ALU + registers.
Symbolic constant: `#define NAME value`.
Short circuit: `&&`/`||` skip the right side when the result is known.
Call by value / reference: copy vs address (C pointers, C++ references).
Prototype: lets the compiler check calls before the definition.
Static local: keeps value between calls.
Recursion vs loop: shorter for recursive problems; uses stack, slower.
Mutual recursion: A calls B, B calls A (needs a prototype).
Memory leak / dangling pointer: not freed / points to freed memory.
Null character: marks the end of a string.
Array of pointers vs 2D array: exact sizes, fast swaps vs fixed size.
Bit-field: member with chosen bits; no address.

Text vs binary file: characters vs raw bytes.
BST: left $<$ node $<$ right; search $\log n$.
Encapsulation: class = data + functions, data private.
Namespace: avoids name clashes; std library in `std`.
Inline: body pasted at call; small functions only.
Virtual destructor: needed for delete via base pointer.
Abstract class: has a pure virtual function, no objects.
Static member: one copy per class; static function has no `this`.
Friend: non-member allowed to use private members.
Can we run C without an editor? Yes, only a compiler is needed (`gcc a.c -o a`).

8.4 Exam-day plan

1. First 5 minutes: read all questions, mark the ones whose type you know (output, bits, switch, file, class).
2. Do output-tracing and bit-pattern parts first — quick, exact marks. Show the working table.
3. For programs: write the **algorithm in 2–3 lines** first, then code using the templates in this guide. Include headers, variable declarations and the I/O format asked.
4. Respect every constraint ("no if-else", "no array", "pointer only", "no string.h", "no VLA").
5. If the paper's sample output looks wrong, say so in one line and solve consistently.
6. Last 10 minutes: check `&` in scanf, `;` after struct/class/do-while, `'\0'`, `free/delete[]`, `fclose`, return types.

You have everything you need. Revise the templates, trust the method, and good luck!